

MapSplat User Guide

MapSplat — User Guide

MapSplat turns the layers in your QGIS project into a self-contained web map — a folder of static files (vector tiles in **PMTiles** format + an **HTML viewer** built on **MapLibre GL JS**) that you can open in any browser or deploy to any static web host. No tile server, no backend.

1. What you need

Requirement	Why	Notes
QGIS 4.0+	The plugin runs inside QGIS	PyQt6 build
GDAL 3.8+	Converts your vector layers to PMTiles	Ships with QGIS; nothing to install
<code>pmtiles</code> CLI	Only for Protomaps basemaps (<i>Download & clip</i>) and raster export	Install from the releases page and put it on your <code>PATH</code> . Core export, XYZ basemaps and streaming do not need it.

You do **not** need Node, a database, or any web stack.

2. Quick start (2 required steps)

Open **Plugins ► MapSplat** (or the toolbar button) to show the dock, then on the **Inputs** tab:

1. ① **Layers** — tick the vector layers you want to publish. Their QGIS **styles and labels are read automatically**. (When you open the dock, your currently-visible layers are pre-selected.)
2. ② **Output** — set a **Project name** and an **Output folder**. The export is written to `<output folder>/<project name>_webmap/`. (These are pre-filled from your project — adjust if needed.)

The **readiness line** above the Export button tells you what's still missing; when it turns green (“*Ready to export*”) the button enables. Click **Export Web Map**, watch the **Log** tab, then **Open Folder** to see the result.

That is the complete required workflow. Everything below is optional and has sensible defaults.

3. The dock, tab by tab

- **Inputs** — Layers, Output, and the Export button. A full run lives here. Type tags show each layer's kind (`[Polygon]` , `[Line]` , `[Point]` , `[VectorTile]` , `[Online]`); an online tag marks layers that stream live and need internet (see [Hosting](#)).
- **Options** — *Export Options* (PMTiles mode, max zoom, tile-count estimate, **Include raster layers**, style.json, export extent) and *Basemap Overlay* (a Protomaps basemap clipped into the export or streamed from your own host, with built-in styles, or an **XYZ raster** provider; the **?** button opens the basemap guide). Defaults are fine for most maps.
- **Viewer** — what the generated web map shows: scale bar, geolocate, fullscreen, coordinate/zoom readouts, reset/north buttons, label placement, legend, attribution, map dimensions, and the optional on-map **tools** — *Measure* (distance/area), *Draw/sketch* (export GeoJSON/KML), and *Export* (save the map as JPG/PDF). Tools are **off by default**; enable the ones you want.
- **Offline** — bundle MapLibre/PMTiles JS + CSS into the export so the viewer works with no internet.
- **Log** — progress, messages, the export **summary**, and the **version stamp** (bottom-right).

The **layer order and groups in the web map follow your QGIS layer tree** in both PMTiles modes — arrange your layers (and groups like “My Layers”) in QGIS and the exported map's stacking and layer list match, with groups and ungrouped layers in the same order as the QGIS panel. A rule-based style's per-rule scale ranges carry over too, so zoom-dependent rules appear only in their band. In the viewer, each layer and each group has an on/off checkbox; the basemap and vector-tile bases collapse into their own sections at the bottom.

4. Key options explained

- **PMTiles mode** — *Single file* merges all layers into one `.pmtiles` ; *Separate files* writes one per layer (loaded independently in the viewer).
 - **Max zoom** — higher = more detail but **exponentially** more tiles/time. 6–10 suits most maps; 14+ can take a long time on large data. The live estimate under it shows the rough tile count/size.
 - **Include raster layers** (*Export Options*) — off by default. Tiles selected local rasters (imagery, scanned maps) to PMTiles; needs GDAL's MBTiles driver. See [Limitations](#).
 - **Export extent** — clip the basemap to a chosen layer's extent or the current map view instead of the full data extent.
 - **Verify PMTiles after export** (*Advanced*) — runs `pmtiles verify` on each written file.
 - **Refresh / Clear basemap cache** (*Advanced*) — basemap extracts are cached; force a re-download or free disk space.
 - **Style only** (*Advanced*) — regenerate `style.json` + the viewer without re-tiling the data.
-

5. Adding a basemap (optional)

A basemap gives your data context (streets, water, terrain). You have two options — a **Protomaps** PMTiles basemap (below), or an **XYZ raster** provider (OpenStreetMap, Carto, OpenTopoMap, Esri World Imagery, or a custom `{z}/{x}/{y}` URL) chosen on **Options ► Basemap Overlay ► XYZ raster**. The XYZ option streams live and needs no `pmtiles` CLI, but the map then needs internet — see [Basemaps](#) and [Hosting](#).

Protomaps basemap

MapSplat uses **Protomaps**: free, OpenStreetMap-derived vector basemaps in PMTiles format. Click the **?** button beside **Basemap Overlay** for the full step-by-step basemap guide (bundled with the plugin; also online at johnzastrow.github.io/mapsplat4/basemaps.html).

Recommended: Download & clip offline. On **Options ► Basemap Overlay**, enable the basemap, choose **Download & clip offline**, and click **Latest** to fill in the newest Protomaps daily planet build. At export MapSplat runs `pmtiles extract --bbox` to clip just your area into the export, then overlays your layers on top. **Copy extract command** gives you the same command to run yourself, so you can clip once and reuse the file via **Source: Local file**.

Styles. Pick a **Basemap style**: five Protomaps styles (Light, Dark, White, Grayscale, Black) ship with MapSplat, so no file is needed. **Custom style.json...** accepts any Protomaps-compatible MapLibre style, for example from *Get style JSON* on maps.protomaps.com. The built-in styles load fonts and icons from `protomaps.github.io` when the map is viewed.

Streaming. *Stream from URL* reads a PMTiles file live, so it needs a file **you host with CORS enabled**. Protomaps' daily builds only allow their own viewer to read them from a browser, so they cannot be streamed; MapSplat warns if you try, and **Test** checks any URL.

Download & clip needs the `pmtiles` CLI. The clip shells out to the `pmtiles` command. MapSplat does not bundle that program (QGIS forbids shipping executables in plugins), so install it once from the [go-pmtiles releases](#) and put it on your `PATH`; the Options tab shows a note with the link if it is missing. Exporting *your* layers uses GDAL and needs **no** CLI; only the basemap clip does.

6. Viewing / serving the output

The export folder contains `index.html`, a `data/` folder of `.pmtiles`, and (optionally) a `lib/` folder of bundled JS/CSS. Because PMTiles uses HTTP **Range** requests, opening `index.html` directly from disk may not work — serve it over HTTP:

- **Bundled dev server:** run `serve.py` in the export folder (`python serve.py`) and open the printed URL.

- **Any static host** that supports range requests: Netlify, Cloudflare Pages, S3/CloudFront, GitHub Pages, nginx, Caddy, etc. Just upload the folder.

Caching: keep re-exports fresh

`serve.py` sends `Cache-Control: no-store` so re-exporting a map always shows the latest result. On a production host you want the same guarantee — otherwise, because MapSplat **overwrites files in place** (same `index.html` / `style.json` / `.pmtiles` names each export), a browser can serve a **stale** cached copy and a changed or added layer will look “missing” until a hard refresh.

Caddy serves HTTP Range requests natively (so PMTiles are served correctly); add cache headers to match:

```
map.example.com {
  root * /var/www/mymap_webmap
  file_server                # Range requests supported by default (PMTiles OK)

  # Mirror serve.py – never serve a stale export
  header Cache-Control "no-store, no-cache, must-revalidate, max-age=0"
  header Pragma "no-cache"
  header Expires "0"
}
```

If you prefer to cache the heavy, rarely-changing assets for speed and only force-revalidate the small files that change every export, no-store just the HTML/JSON and long-cache the rest:

```
map.example.com {
  root * /var/www/mymap_webmap
  file_server

  @fresh path *.json /index.html / # style.json + the viewer must always be current
  header @fresh Cache-Control "no-store"

  @assets path /data/* /lib/* /patterns/* *.png # tiles, libs, sprites, hatches
  header @assets Cache-Control "public, max-age=3600, must-revalidate"
}
```

Note: MapSplat reuses filenames across exports, so long-lived caching of `/data/*` risks stale tiles after a re-export. Use the second config only if you redeploy to a **fresh directory** (or purge the CDN) on each publish; otherwise prefer the first (no-store everything).

nginx equivalent: `add_header Cache-Control "no-store";` in the `location /` block (nginx serves Range requests by default).

7. Troubleshooting

- “**pmtiles CLI not found**” — only needed for the basemap. Install it and ensure QGIS sees your `PATH`. If it works in a terminal but not from the QGIS *Python Console* (`shutil.which("pmtiles")` returns `None`), launch QGIS from a terminal so it inherits your full shell `PATH`.
 - **Export is huge / slow** — lower **Max zoom**; watch the tile estimate.
 - **Blank map in the browser** — you opened `index.html` from disk; serve it over HTTP (§6).
 - **The dock looks unchanged after an update** — QGIS caches plugin code; **fully restart QGIS** (or use *Plugin Reloader*). Confirm via the **version stamp** on the Log tab.
 - **A layer’s symbology didn’t translate** — a warning icon in the layer list flags renderers/markers (heatmap, point cluster, font markers...) that don’t map cleanly to MapLibre.
-

8. Styling — what carries over, and what doesn’t

MapSplat reads each layer’s QGIS symbology and labels and converts them to a MapLibre style. Most everyday styling translates well; a few QGIS features have no MapLibre equivalent. Layers with symbology that won’t translate cleanly are flagged with a warning icon in the layer list (hover for why).

Translates well - Single-symbol, categorized, graduated, and rule-based renderers. - Fill and stroke colours, widths, opacity, and simple line/dash patterns. - **SVG markers** — converted to a sprite sheet — and basic marker/line/fill symbols. - Labels: text, font, size, colour, and halo.

Limited or not supported - **Heatmap renderer** — exported as circle markers, not a smooth heatmap. - **Point cluster renderer** — clustering isn’t reproduced; points render at their true positions. - **Point displacement renderer** — displaced positions aren’t preserved. - **Font markers** — render as a plain circle (use an SVG marker for a custom glyph). - **Draw effects** (drop shadow, glow, blur), **blend modes**, the **2.5D** renderer, and **geometry generators** — no MapLibre equivalent; ignored. - **Data-defined (expression) overrides** on symbol properties — only simple cases translate. - Very complex or deeply nested rule sets may be simplified.

Tip: for the most faithful web map, favour categorized / graduated / rule renderers with solid fills, strokes, and SVG markers, and keep data-defined symbology simple.

9. Versions this build was made with

Component	Version
MapSplat	0.45.0
MapLibre GL JS (viewer)	5.24.0
PMTiles JS (viewer)	4.4.1
<code>pmtiles</code> CLI (basemap only; tested)	1.30.1

Component	Version
QGIS	4.0+ required — built and tested on 4.2
GDAL	3.8+ required (PMTiles driver) — tested on 3.12
Qt / Python bindings	PyQt6 (QGIS 4)

The MapLibre and PMTiles **JS** versions are pinned in the generated viewer; the `pmtiles` **CLI** is whatever you have installed on your `PATH` .

MapSplat is free software (GPL-2.0-or-later). Source, issues, and updates: <https://github.com/johnzastrow/mapsplat4>.